

Avaliação 3 – Relatório de laboratório com análise de tráfego real e configuração de rede em ambiente simulado

Ficha Técnica

Item	Descrição
Disciplina	Redes de Computadores
Ambiente	Desktops com Windows 11 + WSL2 (Ubuntu 24.04), com acesso à Internet
Ferramentas	ip (iproute2), ping , traceroute , tc , iperf3 , tcpdump , Python 3
Nota	25 pontos

Importante a soma das questões é 100 pontos, para facilitar a correção. A nota final será calculada pela fórmula:
$$NotaFinal = (\sum_1^{31} Questões) * 0,25$$

A qualidade do documento final será considerada.

Nota de ambiente: o laboratório usa *network namespaces* do Linux para transformar o próprio Ubuntu/WSL em uma mini-rede virtual com um roteador, dois hospedeiros de origem e quatro "vizinhos" de destino. Nenhum equipamento físico é necessário e nada é alterado na rede real do laboratório. Todos os comandos de configuração afetam apenas os namespaces virtuais criados na prática.

O que deve ser entregue?

Cada dupla deve entregar um arquivo **.zip** contendo:

- 1. Documento PDF seguindo estritamente o Modelo_relatório_avaliação_3, em conformidade com a estrutura obrigatória do relatório
- 2. Códigos e scripts
- 3. Arquivos de capturas de tráfego (.pcap)

Estrutura Obrigatória do Relatório

Seção	Conteúdo
Capa	Nomes, matrículas, número da dupla, data
Síntese (Resumo Executivo)	De 1 a 2 páginas explicando o que foi realizado , a metodologia, os principais resultados e dificuldades encontradas
Respostas das questões	Respostas numeradas exatamente como no roteiro (Q1, Q2, ...), com evidências (saída de terminal em bloco de código, com date; whoami visíveis)
Apêndice A: Capturas de dados	Lista dos arquivos <code>.pcap</code> , filtros utilizados e <i>prints</i> da análise
Referências	Fontes consultadas (inclusive ferramentas de IA, se houver)

Grade de correção**Eixo de Corretude Técnica (100 Pontos Base)**

Este eixo avalia as respostas e os resultados técnicos exigidos no guião.

Secção Temática	Questões e Pontuações	Critérios de Correção a Observar	Pontuação Máxima
Configuração e Repasse	Q1 (2), Q2 (2), Q3 (2), Q4 (2)	Identificação correta de IP, <i>gateway</i> , namespaces, versão do WSL e compreensão inicial do repasse (LPM).	8 pontos
LPM e Roteamento	Q5 (3), Q6 (2), Q7 (2), Q8 (2), Q9 (6), Q10 (3)	Preenchimento correto da tabela (Q5), comportamento de rotas específicas/default (Q6-Q8), execução exata do script <code>lpm.py</code> e compreensão do SDN (Q9-Q10).	18 pontos
QoS, Enfileiramento e HOL	Q11 (3), Q12 (8), Q13 (3), Q14 (4), Q15 (6)	Dados consistentes no <code>iperf3</code> /ping, compreensão clara do <i>buffer overflow</i> , cálculos precisos do <i>fabric</i> ($N \times R$) e análise do simulador HOL (<i>FIFO</i> vs <i>VOQ</i>).	24 pontos
Estrutura IPv4 e Fragmentação	Q16 (2), Q17 (2), Q18 (7), Q19 (2), Q20 (2), Q21 (2)	Leitura correta do MTU, validação da topologia e preenchimento exato da tabela do cabeçalho IPv4 (TTL, Protocolo, Checksum).	17 pontos
Cálculos e Capturas de Fragmentação	Q22 (2), Q23 (4), Q24 (4)	Cálculos manuais e de script consistentes com os tamanhos capturados (regra dos 8 bytes), identificação da <i>flag</i> MF e do <i>offset</i> .	10 pontos
PMTUD e ICMP	Q25 (2), Q26 (2), Q27 (4), Q28 (4)	Compreensão da <i>flag</i> DF, interpretação das mensagens ICMP (Type 3) reportadas no <i>tracpath</i> e <i>ping</i> .	12 pontos
Análise Crítica e Segurança	Q29 (3), Q30 (3), Q31 (5)	Explicação sólida sobre os riscos de desempenho, pesquisa adequada sobre ataques (com referências) e cálculo exato do <i>checksum</i> hexadecimal.	11 pontos
Total			100 pontos

Eixo de Qualidade do Documento e Entregáveis (Penalizações)

Critério de Qualidade	Falha / Omissão	Penalização
Formato de Entrega	Não entrega em formato <code>.zip</code> ou arquivo corrompidos.	- 10 pontos
Ficheiros de Código e Capturas	Ausência dos <i>scripts</i> Python (<code>lpm.py</code> , <code>hol_sim.py</code> , <code>frag.py</code>) ou dos <code>.pcap</code> .	- 5 pontos (por arquivo em falta)
Síntese (Resumo Executivo)	Ausência de síntese de 1 a 2 páginas, ou síntese sem a descrição da metodologia e dificuldades.	- 15 pontos
Evidências de Autenticidade	Blocos de código/saídas de terminal sem o <code>date</code> e o <code>whoami</code> visíveis, levantando dúvidas sobre a autoria.	- 20 pontos
Apêndice e Referências	Falta do Apêndice A com os filtros utilizados ou falta de referências bibliográficas (incluindo IA).	- 10 pontos

Eixo de Prazo de Entrega e Ética

Critérios aplicados à gestão da entrega da avaliação:

- **Entrega no Prazo (100%):** Submissão até à data limite estipulada.
- **Entrega com Atraso:** Penalização de 10 pontos por dia de atraso.
- **Corretude Ética:** O uso não referenciado de IA para gerar o relatório ou indícios de plágio/cópia entre grupos resulta na anulação da avaliação (0 pontos).

Roteiro de Laboratório – Aula módulo 4.1

Repasse (*Forwarding*) vs. Roteamento (*Routing*), Longest Prefix Match e Estrutura Interna de um Roteador

Objetivos de Aprendizagem

Ao final do laboratório, o aluno será capaz de:

1. Diferenciar, em um sistema real, o **Plano de Controle** (quem constrói a tabela) do **Plano de Dados** (quem consulta a tabela para cada pacote).
2. Verificar experimentalmente o **Longest Prefix Match (LPM)** com a tabela de repasse do exercício explicada na Aula módulo 4.1.
3. Observar como uma mudança no plano de controle (inserir/remover rota) altera imediatamente o repasse.
4. Relacionar **enfileiramento na porta de saída**, descarte de pacotes e atraso ao tamanho do *buffer*.
5. Demonstrar por simulação o **bloqueio de cabeça de linha (HOL Blocking)** e o ganho das filas virtuais de saída (**VOQ**).
6. Dimensionar a taxa mínima de um *switching fabric* ($N \times R$).

Preparação e Verificação do Ambiente

Instalar os pacotes necessários no Ubuntu

No terminal Ubuntu (WSL):

```
sudo apt update
sudo apt install -y iproute2 iputils-ping iputils-tracepath traceroute tcpdump iperf3 python3
sudo apt install -y tshark
mkdir -p ~/lab4 && cd ~/lab4
```

Na instalação do `tshark`, responda **Não** quando perguntar se usuários comuns podem capturar pacotes; use `sudo`.

Verificar suporte do kernel

```
sudo ip netns add teste && sudo ip netns del teste && echo "netns OK"
sudo tc qdisc show | head -3
```

Resultado esperado: a mensagem `netns OK` e a listagem de `qdiscs` sem erro.

Verificar se o WSL está na versão 2

No **PowerShell** do Windows:

```
wsl -l -v
```

A coluna `VERSION` da distribuição Ubuntu-24.04 deve mostrar **2**. *Network namespaces* e `tc` exigem o WSL2.

No **Terminal do Ubuntu (WSL)**

```
date
whoami
lsb_release -a
uname -r
ip -br addr
ip route
which ping tcpdump tshark python3 ip
```

RESPONDA:

Q1 (2 pts):

- Apresente a saída dos comandos acima e identifique: versão do Ubuntu, versão do *kernel*, endereço IP da interface do WSL, e **gateway padrão** (rota `default`) com a interface associada.
- A saída de `ip route` do WSL contém uma entrada `default`. Explique, com suas palavras, a função dessa linha em termos de **LPM**: em que situação ela é escolhida e por que ela é a rota de prefixo mais curto possível.

Parte 1 – Plano de Controle vs. Plano de Dados no seu sistema

Objetivo: identificar, em um computador real, a *tabela de repasse* e a consulta feita para cada destino.

1.1 Observar a tabela de repasse do WSL

```
ip -4 addr show
ip -4 route show
```

Identifique a **rota padrão** (`default via ...`) e a rota da rede diretamente conectada.

1.2 Consultar como o núcleo decidiria o repasse

```
ip route get 8.8.8.8
ip route get 127.0.0.1
ip route get $(ip -4 route show default | awk '{print $3}')
```

Cada comando executa exatamente uma consulta LPM na tabela (a operação do **plano de dados**), retornando a interface de saída e o próximo salto.

1.3 Observar o caminho fim a fim

```
tracert -n -m 12 8.8.8.8
```

Asteriscos (* * *) são normais: muitos roteadores não respondem às sondas. O primeiro salto tipicamente é o host Windows (WSL2 usa NAT por padrão). Cada linha é um roteador que fez **repasse** local; ninguém "conhece" o caminho inteiro.

RESPONDA

Q2 (2 pts). Quantos saltos foram visíveis no `traceroute` ? Isso confirma que o roteador decide o caminho completo ou apenas a **próxima porta de saída**? Justifique com a distinção entre repasse e roteamento.

Q3 (2 pts). Classifique como **Plano de Dados** ou **Plano de Controle**:

- a) `ip route get`
- b) `ip route add`
- c) recebimento de um anúncio OSPF
- d) escolha da interface de saída de um datagrama.

Parte 2 – Construção do Roteador Virtual

2.1 Topologia

```

172.16.1.0/24                                10.0.0.0/30  [n0] Interface 0 (default)
[h1] .2 ----- .1 \                        / 10.0.1.0/30  [n1] Interface 1
                                [  r  (roteador) ]-- 10.0.2.0/30  [n2] Interface 2
[h2] .2 ----- .1 /                        \ 10.0.3.0/30  [n3] Interface 3
172.16.2.0/24
```

Namespace	Papel	Endereço(s)
r	Roteador (encaminhamento IPv4 ativo)	172.16.1.1, 172.16.2.1, 10.0.i.1 (i = 0..3)
h1 , h2	Hospedeiros de origem	172.16.1.2 e 172.16.2.2
n0 ... n3	"Vizinhos" de saída (Interface 0 a 3)	10.0.i.2

Para tornar o LPM visível, **os quatro vizinhos** possuem, na interface *loopback*, os mesmos três endereços de teste:

192.168.21.15 , 192.168.18.200 e 10.0.0.5 . Assim, o **roteador** decide qual vizinho receberá o pacote, e o contador de pacotes de cada interface do roteador revela a escolha.

2.2 Script de criação

Crie o arquivo `setup.sh` (`nano setup.sh`), cole o conteúdo abaixo, salve e execute `sudo bash setup.sh` :

```
#!/bin/bash
set -e

# Limpar o ambiente
for ns in r h1 h2 n0 n1 n2 n3; do sudo ip netns del $ns; done

ip netns add r
for h in h1 h2; do ip netns add $h; done
for i in 0 1 2 3; do ip netns add $i; done

ip netns exec r sysctl -qw net.ipv4.ip_forward=1
ip -n r link set lo up

# Hospedeiros de origem
for k in 1 2; do
    ip link add h$k-r type veth peer name r-h$k
    ip link set h$k-r netns h$k
    ip link set r-h$k netns r
    ip -n h$k addr add 172.16.$k.2/24 dev h$k-r
    ip -n r addr add 172.16.$k.1/24 dev r-h$k
    ip -n h$k link set lo up
    ip -n h$k link set h$k-r up
    ip -n r link set r-h$k up
    ip -n h$k route add default via 172.16.$k.1
done

# Vizinhos de saída (Interfaces 0 a 3)
for i in 0 1 2 3; do
    ip link add n$i-r type veth peer name r-n$i
    ip link set n$i-r netns n$i
    ip link set r-n$i netns r
    ip -n n$i addr add 10.0.$i.2/30 dev n$i-r
    ip -n r addr add 10.0.$i.1/30 dev r-n$i
    ip -n n$i link set lo up
    ip -n n$i link set n$i-r up
    ip -n r link set r-n$i up
    ip -n n$i route add default via 10.0.$i.1
    for a in 192.168.21.15 192.168.18.200 10.0.0.5; do
        ip -n n$i addr add $a/32 dev lo
    done
done

echo "Topologia criada."
```

2.3 Funções auxiliares

Cole no terminal (valem para toda a sessão):

```

cont() {
    for i in 0 1 2 3; do
        printf "Interface %s: %s pkts TX | " $i \
            $(sudo ip -n r -s link show dev r-n$i | awk '/TX:/{getline; print $2}')
        done; echo
    }
    alias R='sudo ip netns exec r'
    alias H1='sudo ip netns exec h1'
    alias H2='sudo ip netns exec h2'
    shopt -s expand_aliases

```

2.4 Validação

```

sudo ip netns list
sudo ip -4 -n r addr show | grep inet
H1 ping -c 2 172.16.1.1
sudo ip -n r route show

```

Resultado esperado: seis namespaces listados; o ping ao roteador responde; a tabela de r contém apenas rotas das redes **diretamente conectadas** (nenhuma delas cobre 192.168.x.x).

RESPONDA:

Q4 (2 pts). Se, agora, executar `H1 ping -c 1 192.168.21.15`, o que deve ocorrer e por quê? Teste e registre a mensagem.

Parte 3 – Longest Prefix Match na prática

Vamos inserir na tabela as rotas apresentadas **na aula do módulo 4.1**.

Entrada	Prefixo	Interface de saída (vizinho)
A	192.168.0.0/16	Interface 1 (n1, 10.0.1.2)
B	192.168.16.0/20	Interface 2 (n2, 10.0.2.2)
C	192.168.20.0/22	Interface 3 (n3, 10.0.3.2)
D	0.0.0.0/0 (padrão)	Interface 0 (n0, 10.0.0.2)

3.1 Instalar a tabela de repasse (ação do plano de controle, feita manualmente)

```

R ip route add 192.168.0.0/16 via 10.0.1.2
R ip route add 192.168.16.0/20 via 10.0.2.2
R ip route add 192.168.20.0/22 via 10.0.3.2
R ip route add default via 10.0.0.2
R ip route show

```

3.2 Prever antes de testar

RESPONDA:

Q5 (3 pts). ANTES de executar, preencha a coluna "Previsão" da tabela do seu relatório:

Pacote	IP de destino	Entradas que casam	Previsão (entrada / interface)
1	192.168.21.15	?	?

Pacote	IP de destino	Entradas que casam	Previsão (entrada / interface)
2	192.168.18.200	?	?
3	10.0.0.5	?	?

3.3 Verificar a decisão do roteador (consulta LPM)

```
R ip route get 192.168.21.15
R ip route get 192.168.18.200
R ip route get 10.0.0.5
```

3.4 Verificar com tráfego real

Compare os contadores da porta de saída antes e depois de enviar 10 pacotes. O ping de um datagrama vindo de h1 percorre $h1 \rightarrow r \rightarrow$ vizinho escolhido:

```
cont
H1 ping -c 10 -i 0.2 192.168.21.15
cont
```

Repita para os outros dois destinos (192.168.18.200 e 10.0.0.5), sempre registrando cont antes e depois.

As interfaces que **aumentaram cerca de 10 pacotes** (mais poucos pacotes de ARP) indicam a porta de saída escolhida pelo repasse.

Opcionalmente, confirme visualmente com tcpdump em duas janelas (terminais):

```
# Terminal 1
R tcpdump -ni r-n3 icmp
# Terminal 2
H1 ping -c 3 192.168.21.15
```

3.5 Experimentos de mudança da tabela

Experimento A – Remover a rota mais específica (C)

```
R ip route del 192.168.20.0/22
R ip route get 192.168.21.15
```

RESPONDA:

Q6 (2 pts). Para onde vai agora o pacote 1? Qual entrada passou a valer?

Experimento B – Inserir uma rota ainda mais específica (/25)

```
R ip route add 192.168.18.128/25 via 10.0.1.2
R ip route get 192.168.18.200
R ip route get 192.168.18.100
```

RESPONDA:

Q7 (2 pts). Por que os dois destinos, ambos em 192.168.18.x, têm decisões diferentes?

Experimento C – Remover a rota padrão


```
R ip route del default
H1 ping -c 2 10.0.0.5
```

RESPONDA:

Q8 (2 pts). Registre a mensagem de erro. O que acontece com um datagrama quando nenhuma entrada casa?

Restaurar o estado original antes de continuar:

```
R ip route del 192.168.18.128/25
R ip route add 192.168.20.0/22 via 10.0.3.2
R ip route add default via 10.0.0.2
```

3.6 Implementando o LPM em Python (comparar com o kernel)

Crie o arquivo `lpm.py` :

```
#!/usr/bin/env python3
import ipaddress, sys

TABELA = [
    ("192.168.0.0/16", "Interface 1"),
    ("192.168.16.0/20", "Interface 2"),
    ("192.168.20.0/22", "Interface 3"),
    ("0.0.0.0/0", "Interface 0"),
]

def lpm(destino):
    d = ipaddress.ip_address(destino)
    candidatos = [(ipaddress.ip_network(p), i) for p, i in TABELA
                  if d in ipaddress.ip_network(p)]
    if not candidatos:
        return None, None
    rede, iface = max(candidatos, key=lambda c: c[0].prefixlen)
    return rede, iface

for ip in sys.argv[1:]:
    d = ipaddress.ip_address(ip)
    print(f"{ip}>16} bits: {int(d):032b}")
    rede, iface = lpm(ip)
    print(f"{'':>16} -> {rede} ({rede.prefixlen} bits) => {iface}")
```

Execute:

```
python3 lpm.py 192.168.21.15 192.168.18.200 10.0.0.5
```

RESPONDA:**Q9 (6 pts):**

- Os resultados do script coincidem com `ip route get` ?
- No script, o que representa a função `max(..., key=prefixlen)` no hardware de um roteador real? Qual componente da Aula módulo 4.1 realiza essa busca em um ciclo de relógio?
- Adapte a `TABELA` do script para incluir a rota `192.168.18.128/25 -> Interface 1` e teste `192.168.18.200` .

Parte 4 – Plano de Controle em Ação

Em roteadores reais, um algoritmo (OSPF, BGP) ou um controlador SDN muda a tabela **enquanto** o plano de dados continua encaminhando. Vamos simular essa dinâmica.

4.1 Mudança de rota durante um fluxo contínuo

Janela 1 – tráfego contínuo em h1 (deixar rodando):

```
H1 ping -i 0.5 192.168.18.200
```

Janela 2 – roteador como "plano de controle":

```
cont
R ip route replace 192.168.16.0/20 via 10.0.1.2
sleep 3
cont
R ip route replace 192.168.16.0/20 via 10.0.2.2
```

Depois, pare o ping na Janela 1 (Ctrl+C).

RESPONDA:

Q10 (3 pts):

- a) Após `ip route replace`, qual interface passou a receber os pacotes de 192.168.18.200? O ping foi interrompido?
- b) Na analogia da aula (guarda de trânsito vs. GPS), qual comando representa o GPS e qual representa o guarda?
- c) Cite uma vantagem e um risco de o plano de controle ser **centralizado** (SDN) em comparação com o tradicional (por roteador).

Parte 5 – Enfileiramento na Porta de Saída

Objetivo: provocar o enfileiramento que ocorre quando duas entradas competem pela mesma porta de saída, e medir o impacto do tamanho do *buffer*.

5.1 Limitar a porta de saída (Interface 1) a 10 Mbit/s

```
R tc qdisc add dev r-n1 root tbf rate 10mbit burst 16kb limit 30kb
R tc qdisc show dev r-n1
```

Aqui o `limit` faz o papel do **buffer de saída** (30 KB $\approx$ 24 ms de atraso máximo a 10 Mbit/s).

Se o `tc` reportar erro de *qdisc* não suportada. O restante do laboratório (Partes 1 a 4 e 6) não depende dessa parte.

5.2 Servidores no vizinho n1

```
sudo ip netns exec n1 iperf3 -s -p 5201 -D
sudo ip netns exec n1 iperf3 -s -p 5202 -D
```

5.3 Experimento 1 – Buffer pequeno (30 KB)

Duas "portas de entrada" (h1 e h2) enviam 8 Mbit/s cada, somando **16 Mbit/s** para uma saída de **10 Mbit/s**.

Execute em ordem (o comando `wait` aguarda os fluxos terminarem):

```
H1 iperf3 -c 10.0.1.2 -p 5201 -u -b 8M -t 10 > h1_buf30k.txt &
H2 iperf3 -c 10.0.1.2 -p 5202 -u -b 8M -t 10 > h2_buf30k.txt &
sleep 3
H1 ping -c 5 -i 0.5 10.0.1.2
wait
grep -E "sender|receiver" h1_buf30k.txt h2_buf30k.txt
R tc -s qdisc show dev r-n1
```

Registre:

- a) taxa recebida somada
- b) percentual de perda em cada fluxo
- c) RTT médio do ping durante a carga
- d) contador dropped do tc

5.4 Experimento 2 – Buffer grande (500 KB)

```
R tc qdisc replace dev r-n1 root tbf rate 10mbit burst 16kb limit 500kb
H1 iperf3 -c 10.0.1.2 -p 5201 -u -b 8M -t 10 > h1_buf500k.txt &
H2 iperf3 -c 10.0.1.2 -p 5202 -u -b 8M -t 10 > h2_buf500k.txt &
sleep 3
H1 ping -c 5 -i 0.5 10.0.1.2
wait
grep -E "sender|receiver" h1_buf500k.txt h2_buf500k.txt
R tc -s qdisc show dev r-n1
```

5.5 Experimento 3 – Sem congestionamento (controle)

Com o buffer de 500 KB, execute somente **um** fluxo de 8 Mbit/s (h1) e compare com os anteriores:

```
H1 iperf3 -c 10.0.1.2 -p 5201 -u -b 8M -t 5 | grep -E "sender|receiver"
```

RESPONDA:

Q11 (3 pts). Preencha a tabela abaixo com os resultados obtidos.

Cenário	Carga oferecida	Perda total (%)	RTT do ping sob carga (ms)
Buffer 30 KB, 2 fluxos	16 Mbit/s		
Buffer 500 KB, 2 fluxos	16 Mbit/s		
Buffer 500 KB, 1 fluxo	8 Mbit/s		

RESPONDA:

Q12 (8 pts):

- a) Por que a perda ocorre mesmo com o roteador "rápido"? Relacione com a Aula módulo 4.1 (múltiplas entradas para a mesma porta de saída, *buffer overflow*).
- b) Como o tamanho do buffer troca **perda** por **atraso**? Qual configuração é melhor para tráfego interativo? E para transferência de arquivos?
- c) Estime a perda teórica mínima quando 16 Mbit/s são oferecidos a um enlace de 10 Mbit/s. O resultado medido se aproxima?
- d) Como o TCP reagiria a esses descartes? Qual mecanismo reduziria a carga oferecida?

Parte 6 – HOL Blocking, VOQ e Dimensionamento do Fabric

6.1 Cálculo do meio de comutação

Para um roteador com N portas de entrada e taxa de linha R , a taxa mínima do fabric que evita enfileiramento na entrada é:

$$R_{fabric} = N \times R$$

RESPONDA:

Q13 (3 pts):

- a) Para $N = 8$ e $R = 10$ Gbps, qual é a taxa mínima do fabric?
- b) Para $N = 32$ portas de 100 Gbps, qual é a taxa mínima? Compare-a com a capacidade de um barramento compartilhado (que comuta **um** pacote por vez) de 400 Gbps e explique por que a arquitetura *crossbar* é necessária.
- c) Em qual das três arquiteturas (memória, barramento, *crossbar*) o seu WSL está, ao repassar pacotes entre namespaces? Justifique (dica: quem copia o pacote?).

6.2 Simulação de HOL Blocking

O bloqueio de cabeça de linha ocorre quando o primeiro pacote da fila de entrada aguarda uma saída ocupada, e trava os demais. O simulador abaixo modela um comutador $N \times N$ com **carga saturada** (sempre há pacotes em todas as entradas, com destinos aleatórios uniformes) em dois modos:

- **FIFO**: cada entrada tem uma única fila (sofre HOL).
- **VOQ**: cada entrada mantém uma fila virtual por saída, e um emparelhamento tenta usar todas as saídas livres.

Crie o arquivo `hol_sim.py` :

```
#!/usr/bin/env python3
"""Simulador de computador com filas na entrada: FIFO (HOL) vs VOQ. Carga saturada."""
import random, sys

def simular(N, slots, modo, seed=1):
    rnd = random.Random(seed)
    enviados = 0
    if modo == "fifo":
        cabeca = [rnd.randrange(N) for _ in range(N)]
        for _ in range(slots):
            pedidos = {}
            for i, d in enumerate(cabeca):
                pedidos.setdefault(d, []).append(i)
            for d, entradas in pedidos.items():
                vencedora = rnd.choice(entradas)
                cabeca[vencedora] = rnd.randrange(N)
                enviados += 1
    else:
        voq = [[0] * N for _ in range(N)]
        for i in range(N):
            for _ in range(N):
                voq[i][rnd.randrange(N)] += 1
        for _ in range(slots):
            saidas_livres = set(range(N))
            entradas_livres = list(range(N))
            rnd.shuffle(entradas_livres)
            for _rodada in range(N):
                for i in list(entradas_livres):
                    cand = [o for o in saidas_livres if voq[i][o] > 0]
                    if cand:
                        o = rnd.choice(cand)
                        saidas_livres.discard(o)
                        entradas_livres.remove(i)
                        voq[i][o] -= 1
                        enviados += 1
            for i in range(N):
                voq[i][rnd.randrange(N)] += 1
        return enviados / (slots * N)

if __name__ == "__main__":
    slots = int(sys.argv[1]) if len(sys.argv) > 1 else 20000
    print(f"{ 'N':>4} { 'FIFO (HOL)':>12} { 'VOQ':>8}")
    for N in (2, 4, 8, 16, 32):
        print(f"{N:>4} {simular(N, slots, 'fifo'):>12.3f} {simular(N, slots, 'voq'):>8.3f}")
```

Execute o script

```
python3 hol_sim.py 5000
```

Resultado esperado (ordem de grandeza): a vazão com FIFO cai de cerca de 0,75 ($N = 2$) para cerca de 0,59–0,62 para N grande, próxima do limite teórico clássico de $\approx 58,6\%$ para carga uniforme. Com VOQ, fica acima de 0,98.

RESPONDA:

Q14 (4 pts):

- Qual fração da capacidade do fabric é desperdiçada por HOL para $N = 8$?
- Considerando o esquema ilustrativo abaixo, desenhe a sequência de três intervalos de tempo t_0 , t_1 e t_2 . Em qual intervalo o pacote [Pacote → Porta X] é enviado: (1) utilizando FIFO e (2) utilizando VOQ?

Fila de Entrada 1: [Pacote → Porta Y] [Pacote → Porta X] --> bloqueado, esperando Y
 Fila de Entrada 2: [Pacote → Porta Y] -----> usando a Porta Y

* O pacote destinado à Porta X fica retido injustamente, mesmo a Porta X estando livre.

c) Por que aumentar a velocidade do fabric acima de $N \times R$ ou adotar VOQ elimina o problema?

Parte 7 – Limpeza e Fechamento

7.1 Limpar o ambiente

```
sudo pkill iperf3
for ns in r h1 h2 n0 n1 n2 n3; do sudo ip netns del $ns; done
sudo ip netns list
```

A lista deve ficar vazia. Os *veth* são removidos automaticamente junto com os namespaces.

7.2 Compreensão do laboratório

RESPONDA:

Q15 (6 pts):

- Explique, com suas palavras e usando um comando executado no laboratório como exemplo, a diferença entre **repasse** e **roteamento**.
- Qual experimento mostrou mais claramente o princípio do LPM e por quê?
- Relacione uma observação da Parte 5 e outra da Parte 6 com a estrutura interna do roteador (portas de entrada, fabric, portas de saída).

Roteiro de Laboratório – Aula módulo 4.2

Datagrama IPv4, MTU e Fragmentação de Pacotes na Prática

Objetivos de Aprendizagem

Ao final do laboratório, o aluno deverá ser capaz de:

- Construir uma topologia virtual com host, roteador e host usando *network namespaces* do Linux, com **MTUs diferentes** em cada enlace.
- Identificar em capturas de pacotes os campos **Version, IHL, Total Length, Identification, Flags, Fragment Offset, TTL, Protocol e Header Checksum**.
- Prever, por cálculo, o resultado da fragmentação e confirmar o resultado por captura.
- Observar que a **recomposição ocorre apenas no destino**, e que roteadores podem **re-fragmentar** fragmentos.
- Demonstrar o efeito da flag **DF** e o funcionamento do **Path MTU Discovery (PMTUD)**.
- Discutir impactos de desempenho e segurança da fragmentação.

Parte 8 – Topologia do Laboratório da Aula módulo 4.2

Toda a topologia roda dentro do WSL e **não depende da internet**, o que torna os resultados reprodutíveis. Os MTUs são escolhidos de modo que o **Host1 fragmenta** (datagrama maior que 1500) e o **Roteador fragmenta de novo** (1500 → 1000).

```

+-----+ 10.0.1.0/24 +-----+ 10.0.2.0/24 +-----+
|  h1      |-----|  r      |-----|  h2      |
| 10.0.1.2 | v1      v1r | (roteador) | v2r      v2 | 10.0.2.2 |
+-----+ MTU = 1500 +-----+ MTU = 1000 +-----+
                        10.0.1.1 / 10.0.2.1

```

Namespace	Interface	Endereço	MTU
h1	v1	10.0.1.2/24	1500
r	v1r	10.0.1.1/24	1500
r	v2r	10.0.2.1/24	1000
h2	v2	10.0.2.2/24	1000

8.1 Verificação do Ambiente

Abra o Ubuntu e anote o MTU da interface real do WSL:

```

ip -br link show eth0
ip link show eth0 | grep -o "mtu [0-9]*"

```

RESPONDA:

Q16 (2 pts): Qual é o MTU da interface `eth0` do seu WSL? Ele é igual ao MTU padrão do Ethernet (1500)?

Parte 9 – Construção da Topologia

1. Crie os três *namespaces* e os dois enlacs virtuais (veth). Copie o bloco inteiro:

```

# Limpar o ambiente
for ns in h1 h2 r; do sudo ip netns del $ns; done

# 1) Namespaces: h1 (host 1), r (roteador), h2 (host 2)
sudo ip netns add h1
sudo ip netns add r
sudo ip netns add h2

# 2) Enlace A: h1 <-> r
sudo ip link add v1 type veth peer name v1r
sudo ip link set v1 netns h1
sudo ip link set v1r netns r

# 3) Enlace B: r <-> h2
sudo ip link add v2r type veth peer name v2
sudo ip link set v2r netns r
sudo ip link set v2 netns h2

```

2. Configure endereços, MTUs e ative as interfaces:

```

sudo ip netns exec h1 ip addr add 10.0.1.2/24 dev v1
sudo ip netns exec r ip addr add 10.0.1.1/24 dev v1r
sudo ip netns exec r ip addr add 10.0.2.1/24 dev v2r
sudo ip netns exec h2 ip addr add 10.0.2.2/24 dev v2

sudo ip netns exec h1 ip link set v1 mtu 1500
sudo ip netns exec r ip link set v1r mtu 1500
sudo ip netns exec r ip link set v2r mtu 1000
sudo ip netns exec h2 ip link set v2 mtu 1000

for ns in h1 r h2; do sudo ip netns exec $ns ip link set lo up; done
sudo ip netns exec h1 ip link set v1 up
sudo ip netns exec r ip link set v1r up
sudo ip netns exec r ip link set v2r up
sudo ip netns exec h2 ip link set v2 up

```

3. Configure o roteamento (rota padrão nos hosts e encaminhamento IP no roteador):

```

sudo ip netns exec h1 ip route add default via 10.0.1.1
sudo ip netns exec h2 ip route add default via 10.0.2.1
sudo ip netns exec r sysctl -w net.ipv4.ip_forward=1

```

4. Valide a conectividade e confira as configurações:

```

sudo ip netns exec h1 ping -c 2 10.0.2.2
sudo ip netns exec h1 ip -br addr
sudo ip netns exec r ip -br link
sudo ip netns exec r ip link show | grep -E "v1r|v2r" | grep -o "mtu [0-9]*"

```

RESPONDA:

Q17 (2 pts): Cole a saída do ping e confirme os MTUs de v1r e v2r .

Opcional: a sequência acima pode ser salva em topologia.sh e executada com sudo bash topologia.sh .

Parte 10 – Anatomia do Cabeçalho IPv4

10.1 Capturando um ping simples (TTL, Protocol, Checksum)

Abra **dois terminais** do Ubuntu (Terminal A e Terminal B).

Terminal A – captura na interface do roteador voltada para o Host1, com detalhamento máximo (-vv):

```
sudo ip netns exec r tcpdump -n -vv -i v1r icmp
```

Terminal B – envie 2 pings de payload pequeno:

```
sudo ip netns exec h1 ping -c 2 -s 56 10.0.2.2
```

Uma linha típica de captura tem o formato:

```

IP (tos 0x0, ttl 64, id 12345, offset 0, flags [DF], proto ICMP (1), length 84)
  10.0.1.2 > 10.0.2.2: ICMP echo request, id 3, seq 1, length 64

```


RESPONDA:**Q18 (7 pts):** Preencha a tabela de análise com os valores observados:

Campo	Valor observado	Explicação (1 linha)
Version	4	
TTL na ida (em <code>v1r</code>)		
Protocol		O que significa o número?
Total Length (<code>length</code>)		
Identification (<code>id</code>)		
Flags		
Fragment Offset		

RESPONDA:**Q19 (2 pts):** O ping padrão de 56 bytes de dados gera um datagrama de 84 bytes. Mostre a conta.

10.2 – O TTL decrementa a cada salto

Repita a captura, agora nas **duas** interfaces do roteador, em paralelo:*Terminal A:*

```
sudo ip netns exec r tcpdump -n -v -i v1r icmp
```

Terminal B:

```
sudo ip netns exec r tcpdump -n -v -i v2r icmp
```

Terminal C (ou o Terminal A após encerrar): se preferir usar apenas dois terminais, execute as duas capturas em sequência.*Terminal de origem:*

```
sudo ip netns exec h1 ping -c 1 10.0.2.2
```

RESPONDA:**Q20 (2 pts):** Qual o TTL do pacote de ida em `v1r` e em `v2r` ? Houve decremento? Por que o **Header Checksum** também muda de um lado para o outro? (Use `-vv` e compare o campo `cksum` .)

10.3 – Expirando o TTL de propósito

```
sudo ip netns exec h1 ping -c 2 -t 1 10.0.2.2
```

```
sudo ip netns exec h1 traceroute -n 10.0.2.2
```

RESPONDA:**Q21 (2 pts):** Quem respondeu com `Time to live exceeded` ? Qual protocolo transportou essa mensagem de erro? Que endereço aparece como "salto 1" no `traceroute` ?

Parte 11 – Pré-cálculo da Fragmentação

Antes de capturar, você deve **prever** o que ocorrerá. Será enviado o mesmo caso discutido em aula: um datagrama de **4000 bytes** (cabeçalho IP de 20 bytes + 3.980 bytes de payload).

Para o `ping`, o payload IP é composto por **8 bytes de cabeçalho ICMP + dados**. Logo, para obter 3.980 bytes de payload IP usa-se `ping -s 3972` ($3.972 + 8 = 3.980$).

11.1 – Cálculo manual (papel)

Use a fórmula da aula:

$$\text{Payload máx. por fragmento} = \left\lfloor \frac{\text{MTU} - 20}{8} \right\rfloor \times 8$$

Complete as tabelas **antes** de executar a captura.

Enlace A (MTU = 1500) – fragmentação feita pelo Host1:

Fragmento	Payload (B)	Total Length (B)	MF	Offset (bytes/8)
1				
2				
3				

Enlace B (MTU = 1000) – re-fragmentação feita pelo roteador. Cada fragmento recebido de 1500 B ou 1040 B ainda é maior que 1000 B e precisa ser subdividido:

Fragmento	Payload (B)	Total Length (B)	MF	Offset (bytes/8)
1a				
1b				
2a				
2b				
3a				
3b				

Atenção: a flag `MF` do último pedaço de um fragmento intermediário permanece `1`. Somente o último pedaço do último fragmento tem `MF = 0`.

11.2 – Verificação por script

Crie o arquivo `frag.py`:

```
#!/usr/bin/env python3
import sys

def fragmentar(payload, mtu, ihl=20, offset_base=0, mf_final=0):
    """Retorna lista de (total_length, mf, offset_em_unidades_de_8, payload_bytes)."""
    max_payload = ((mtu - ihl) // 8) * 8
    frags, pos = [], 0
    while pos < payload:
        tam = min(max_payload, payload - pos)
        ultimo = (pos + tam) >= payload
        mf = mf_final if ultimo else 1
        frags.append((tam + ihl, mf, (offset_base + pos) // 8, tam))
        pos += tam
    return frags

payload = int(sys.argv[1]) if len(sys.argv) > 1 else 3980
mtus = [int(x) for x in sys.argv[2:]] or [1500, 1000]

atual = [(payload + 20, 0, 0, payload)]
for mtu in mtus:
    print(f"\n=== Enlace com MTU = {mtu} ===")
    novos = []
    for (_, mf, off, tam) in atual:
        if tam + 20 <= mtu:
            novos.append((tam + 20, mf, off, tam))
        else:
            novos += fragmentar(tam, mtu, 20, off * 8, mf)
    atual = novos
    print(f"{'#':>2} {'TotalLen':>9} {'MF':>3} {'Offset':>7} {'Payload':>8}")
    for i, (tl, mf, off, tam) in enumerate(atual, 1):
        print(f"{'i':>2} {'tl':>9} {'mf':>3} {'off':>7} {'tam':>8}")
```

Execute:

```
python3 frag.py 3980 1500 1000
```

RESPONDA:

Q22 (2 pts): Compare a saída do script com a sua tabela manual. Corrija eventuais erros e indique em qual passo (regra dos 8 bytes, cálculo do offset ou flag MF) ocorreu a divergência.

Parte 12 – Captura da Fragmentação

12.1 – Fragmentação feita pelo Host1 (enlace de 1500 B)

Terminal A – captura no lado do Host1 e grava em arquivo:

```
sudo ip netns exec r tcpdump -n -v -i v1r -w enlaceA.pcap icmp
```

Terminal B – envie um único ping de 3.972 bytes de dados, **permitindo** fragmentação (`-M dont` zera o bit DF):

```
sudo ip netns exec h1 ping -c 1 -M dont -s 3972 10.0.2.2
```

Encerre a captura (`Ctrl+C`) e leia o arquivo:

```
sudo tcpdump -n -v -r enlaceA.pcap
```

Procure linhas como:

```
IP (tos 0x0, ttl 64, id 4242, offset 0, flags [+], proto ICMP (1), length 1500) 10.0.1.2 > 10.0.2.2: ICMP echo request
IP (tos 0x0, ttl 64, id 4242, offset 1480, flags [+], proto ICMP (1), length 1500) 10.0.1.2 > 10.0.2.2: ip-proto-1
IP (tos 0x0, ttl 64, id 4242, offset 2960, flags [none], proto ICMP (1), length 1040) 10.0.1.2 > 10.0.2.2: ip-proto-1
```

Atenção: o tcpdump mostra o **offset já multiplicado por 8** (em bytes). O campo gravado no cabeçalho é `offset/8`, ou seja 0, 185 e 370. O Wireshark exibe ambos.

12.2 – Re-fragmentação feita pelo Roteador (enlace de 1000 B)

```
sudo ip netns exec r tcpdump -n -v -i v2r -w enlaceB.pcap icmp &
sleep 1
sudo ip netns exec h1 ping -c 1 -M dont -s 3972 10.0.2.2
sleep 1
sudo pkill tcpdump
sudo tcpdump -n -v -r enlaceB.pcap
```

Resultado esperado: 6 fragmentos de ida no enlace B, com *Total Length* de 996, 524, 996, 524, 996 e 64 bytes, todos com o **mesmo id** do enlace A.

12.3 – Quem recompõe? Captura no destino

Mostre que a recomposição acontece **somente no Host2**:

```
sudo ip netns exec h2 tcpdump -n -i v2 -c 11 icmp &
sleep 1
sudo ip netns exec h1 ping -c 1 -M dont -s 3972 10.0.2.2
```

O ping retorna uma única resposta de 3.980 bytes, mesmo que o enlace só transporte pacotes de até 1.000 bytes: o Host2 remontou o datagrama; o Host1 também remonta a resposta.

12.4 – Análise opcional no Wireshark

Se o Wireshark estiver instalado, abra os arquivos gerados pelo WSL.

Abra `enlaceA.pcap` e `enlaceB.pcap` e aplique o filtro:

```
ip.flags.mf == 1 or ip.frag_offset > 0
```

Use também `icmp` e `ip.id == 0x...` (substitua pelo ID observado). Verifique que o Wireshark sinaliza *Fragmented IP protocol* e remonta o datagrama no último fragmento (*Reassembled IPv4 in frame N*).

RESPONDA:

Q23 (4 pts): Preencha a tabela de conferência:

Enlace	Fragmento	Total Length	id	MF	Offset (campo)	Coincide com a previsão?
A (1500)	1					
A (1500)	2					
A (1500)	3					

Enlace	Fragmento	Total Length	id	MF	Offset (campo)	Coincide com a previsão?
B (1000)	1a ... 3b					

RESPONDA:**Q24 (4 pts):**

- a) O roteador copiou o valor do campo *Identification*? Por que isso é indispensável para a recomposição?
- b) Se um único fragmento se perder no enlace B, o que acontece com o datagrama inteiro? Como isso afeta o desempenho em comparação a um pacote não fragmentado?

Parte 13 – Flag DF e Path MTU Discovery

13.1 – DF = 1 no próprio emissor

```
sudo ip netns exec h1 ping -c 1 -M do -s 3972 10.0.2.2
```

Observe: o Linux informa que a mensagem é longa demais para o MTU local (*Message too long, mtu=1500*); o pacote nem sai do Host1.

13.2 – DF = 1 e descarte pelo roteador

Um pacote de 1.472 bytes de dados gera um datagrama de 1.500 bytes (1472 + 8 + 20), que cabe no enlace A, mas **não cabe** no enlace B (MTU 1000):

```
sudo ip netns exec r tcpdump -n -v -i v1r icmp &
sleep 1
sudo ip netns exec h1 ping -c 1 -M do -s 1472 10.0.2.2
sleep 1
sudo pkill tcpdump
```

Saída esperada do ping : From 10.0.1.1 icmp_seq=1 Frag needed and DF set (mtu = 1000) .

RESPONDA:

Q25 (2 pts): Quem enviou a mensagem de erro? Qual o tipo/código ICMP (Destination Unreachable – *Fragmentation Needed*)? Que informação útil essa mensagem carrega para o emissor?

13.3 – Descobrindo o Path MTU

```
sudo ip netns exec h1 tracepath -n 10.0.2.2
```

RESPONDA:

Q26 (2 pts): Qual pmtu o tracepath reportou ao final? Em qual salto ele foi reduzido?

Em seguida, mostre que o pacote com DF=1 passa a funcionar com o tamanho correto (1000 – 28 = 972 bytes de dados):

```
sudo ip netns exec h1 ping -c 1 -M do -s 972 10.0.2.2
sudo ip netns exec h1 ping -c 1 -M do -s 973 10.0.2.2
```

RESPONDA:**Q27 (4 pts):**

- a) Por que 972 funciona e 973 não? Relacione com os 20 bytes do cabeçalho IP e os 8 bytes do cabeçalho ICMP.

- b) Por que o IPv6 aboliu a fragmentação por roteadores e depende de PMTUD? Qual o problema de firewalls que bloqueiam ICMP?

Parte 14 – Fragmentação na Internet Real

Agora use o acesso à internet. Os resultados dependem da rede e de filtros ICMP, portanto **compare e discuta**.

14.1 – No WSL (Ubuntu)

```
ping -c 3 -M dont -s 3972 8.8.8.8
ping -c 3 -M do -s 1472 8.8.8.8
ping -c 3 -M do -s 1473 8.8.8.8
```

14.2 – No Windows (PowerShell ou CMD)

```
ping -n 3 -l 3000 8.8.8.8
ping -n 3 -f -l 1472 8.8.8.8
ping -n 3 -f -l 1473 8.8.8.8
```

(-l define o tamanho dos dados e -f ativa o bit DF no Windows.)

14.3 – Descobrindo o MTU de caminho na internet

Encontre, por tentativa, o maior valor de -s que funciona com -M do até 8.8.8.8 e calcule: $MTU = \text{valor} + 28$.

RESPONDA:

Q28 (4 pts):

- a) Qual o MTU de caminho obtido? Ele é 1500? Se for diferente, indique possíveis causas (VPN, PPPoE, túneis, Wi-Fi, NAT do WSL).
- b) Pacotes grandes sem DF chegaram ao destino? Mesmo que cheguem, por que a fragmentação é desencorajada na internet moderna?

Parte 15 – Limpeza e Fechamento

15.1 Limpar o ambiente

Remova a topologia (as interfaces veth são removidas junto com os namespaces):

```
sudo pkill tcpdump 2>/dev/null
sudo ip netns del h1
sudo ip netns del r
sudo ip netns del h2
ip netns list
```

A saída de `ip netns list` deve estar vazia. Os arquivos em `~/lab4` (capturas, `frag.py`) podem ser mantidos para o relatório.

Parte 16 — Análise Crítica

RESPONDA:

Q29 (3 pts): Explique por que a fragmentação IPv4 é considerada prejudicial ao desempenho (cite ao menos três razões: perda de um fragmento, custo de CPU, buffers de remontagem, ineficiência por cabeçalhos repetidos).

Q30 (3 pts): Pesquise e explique os ataques Ping of Death e Teardrop: qual falha exploravam, como os fragmentos são manipulados (tamanho, offsets) e como sistemas modernos mitigam. Cite ao menos duas referências confiáveis.

Q31 (5 pts): O cabeçalho abaixo (20 bytes, em hexadecimal, com o campo checksum zerado) corresponde a um fragmento observado na captura do datagrama de Identification 777, Fragment Offset = 185, MF = 1, TTL = 63, Protocol = 1 (ICMP), origem 10.1.1.2, destino 10.2.2.2, Total Length 1500:

4500 05DC 0309 20B9 3F01 0000 0A01 0102 0A02 0202

Responda:

- Interprete cada palavra de 16 bits (campo a campo).
- Calcule o checksum (soma em complemento de 1 das palavras de 16 bits, com carry somado de volta, e complemento final) mostrando as somas parciais.
- Recalcule o checksum para o mesmo cabeçalho com TTL = 64 (como estava antes de cruzar o roteador) e explique a diferença entre os dois resultados.